From Web2 UI to Solana frontend architecture.

A MENTAL MODEL FOR DEVELOPERS HEADING INTO COLOSSEUM.

SESSION GOALS

Why Solana frontend engineering is a different discipline — and why the differences are the point.

What you will leave with

- A clear picture of the **official Solana JS stack**, and when to use each piece.
- The **account model**, from a frontend developer's perspective.
- How **transactions** actually flow from click to confirmation.
- The **key environmental differences** that break Web2 assumptions.

> Every concept in this deck is sourced from official Solana documentation. No speculation, no hype.

The official frontend stack

MODERN PATH

- `@solana/kit` — low-level SDK
- `@solana/client` — JSON-RPC client
- `@solana/react-hooks` — React integration

COMPAT PATH

- `@solana/web3.js` 1.x — the classic SDK
- `@solana/web3-compat` — migration bridge

> For Colosseum: start with **web3.js 1.x + Wallet Adapter** (battle-tested, everywhere). Graduate to `@solana/kit` as your app matures.

Source: solana.com/docs/frontend

The account model in one slide

Every account in Solana has **five canonical fields**:

- `lamports` — SOL balance in lamports (1 SOL = 10^9 lamports)
- `data` — raw bytes the owner program interprets
- `owner` — program authorized to mutate this account
- `executable` — is this account a program?
- `rent_epoch` — bookkeeping for rent

> Only the **owner program** can modify an account's data or debit lamports. A frontend never owns state — it reads, and it submits.

Source: solana.com/docs/core/accounts

A frontend is a witness of on-chain state and a courier of user intent — not the source of truth.

Transactions, atomically

A transaction is a **bundle** of instructions signed by a fee payer.

- If **any** instruction fails, the whole transaction is reverted.
- State changes are atomic. The user pays fees regardless.
- Max transaction size: **1232 bytes**.

> Practical impact: never design a UI that "half-succeeds" on chain. Either a transaction confirms or it doesn't.

Source: solana.com/docs/core/transactions

The confirmation lifecycle

STEP 1 Fetch a fresh `blockhash` and `lastValidBlockHeight`.

STEP 2 Build, sign, submit. Track the signature.

STEP 3 Wait for **confirmed** commitment against `lastValidBlockHeight`.

- Blockhashes are valid ~ **60–90 seconds**.
- If the window closes, the transaction **expires**.
- Your UI must express this explicitly — no infinite spinners.

Source: Confirmation guide

What breaks Web2 intuition

SPEED IS REAL BUT BOUNDED

Target slot time is **~400 ms**. Congestion happens. Priority fees exist for a reason.

PUBLIC RPC IS NOT PRODUCTION

Public Solana RPC endpoints are rate-limited and have **no SLA**.

WALLETS ARE EXTERNAL

Your UI can never sign. It can only **ask**.

FEES ARE PAID EVEN ON FAILURE

Users pay for failed transactions. Your UI must explain this.

Source: Production readiness

HANDS ON

You will build an account viewer that reads live data from Devnet using the official stack.

Assignment 1 — live recap

- Install `@solana/react-hooks` and `@solana/client` from npm.
- Connect to Devnet using the official provider.
- Accept any base58 address.
- Display **SOL balance + the five account fields**.
- Link to Solana Explorer for verification.

> Success criteria live in `assignments/assignment-1.md`. Reviewers pull, run, and verify against Explorer.

Quiz 1 — topics covered

- The official Solana frontend stack and what each package does.
- The five fields every account has, and who can modify them.
- Why transactions are **atomic**, and what "expired" means.
- Why public RPC is not production-grade.

> Full quiz with 10 MCQs, 2 open-ended questions, and answer keys: `quizzes/quiz-1.md`.

END OF LECTURE 01

Next: Wallets & transaction UX.

References

- Solana frontend docs — solana.com/docs/frontend
- Accounts model — solana.com/docs/core/accounts
- Transactions — solana.com/docs/core/transactions
- Confirmation — solana.com/developers/guides/advanced/confirmation
- Production readiness — solana.com/docs/payments/production-readiness